Biblioteca Tecnológica — BACKEND (Java + Spring Boot)

Documento de estudo para a apresentação de Backend. Explica o projeto na ordem em que o computador executa, e depois cada arquivo, linha por linha.

---

1. O que o Backend faz (em uma frase)

É um programa Java que fica "ligado" na porta **8089** do computador, esperando pedidos pela internet (protocolo HTTP). Quando o aplicativo pede, ele **lista**, **busca** ou **cadastra** livros e responde em formato **JSON**.

Os 3 endpoints (as 3 "portas de entrada" da API)

Verbo	Endereço	O que faz	Resposta de sucesso
GET	`/livros`	Lista todos os livros	200 OK
GET	`/livros/{id}`	Busca um livro pelo número	200 OK (ou 404 se não existir)
POST	`/livros`	Cadastra um livro novo	201 Created (ou 400 se os dados forem inválidos)

Só existem dois verbos (GET e POST). Não existe PUT, PATCH nem DELETE.

Códigos HTTP usados

Código	Significado	Quando acontece neste projeto
200	OK	GET deu certo
201	Created (criado)	POST cadastrou o livro
400	Bad Request (pedido ruim)	título vazio, sem disponibilidade, capa inválida, JSON quebrado
404	Not Found (não encontrado)	`GET /livros/99`, o livro 99 não existe

---

2. O caminho de um pedido (a ideia mais importante)

O projeto é dividido em **camadas**. Cada camada tem **uma** responsabilidade. Assim o código fica organizado e fácil de explicar.

```
APLICATIVO (Android)
       |  pedido HTTP (JSON)
       v
[ Controller ]  recebe o pedido e devolve a resposta
       |
       v
[ Service ]     faz o trabalho (as "regras" da aplicação)
       |
       v
[ Repository ]  conversa com o banco de dados
       |
       v
[ Banco H2 ]    guarda os livros (na memória)
```

E existem as classes "de apoio":

Classe	Pacote	Papel
`Livro`	entity	Representa **uma linha da tabela** do banco
`LivroRequest`	dto	O que o aplicativo **envia** (e as regras de validação)
`LivroDTO`	dto	O que a API **devolve**
`LivroMapper`	mapper	**Converte** entre Request, Entity e DTO
`LivroNaoEncontradoException`	exception	Erro "livro não existe"
`ApiExceptionHandler`	exception	Transforma erros em respostas JSON com o código certo
`CargaInicial`	(raiz)	Cadastra os 4 livros quando o servidor liga
`BibliotecaApplication`	(raiz)	O `main`, que liga tudo

Exemplo: o caminho do `POST /livros`

1. O app envia `{"titulo":"O Pequeno Príncipe","disponivel":true,"capa":"principe"}`.
2. O **Controller** recebe, transforma o JSON em um `LivroRequest` e **valida** (`@Valid`).
3. Se algo estiver errado, o **ApiExceptionHandler** responde **400** e o Service nem é chamado.
4. Se estiver certo, o Controller chama o **Service**.
5. O Service pede ao **Mapper** para transformar o `LivroRequest` em `Livro` (Entity).
6. O Service pede ao **Repository** para **salvar** no banco. O banco cria o `id`.
7. O Service transforma o `Livro` salvo em `LivroDTO` (com o id) usando o Mapper.
8. O Controller devolve o `LivroDTO` como JSON, com status **201**.

Exemplo: o caminho do `GET /livros/99`

1. O Controller recebe o `99` e chama `livroService.buscarPorId(99)`.
2. O Service pede ao Repository `findById(99)`. O banco não tem esse livro.
3. O Service **lança** `LivroNaoEncontradoException`.
4. O `ApiExceptionHandler` "pega" essa exception e responde **404** com `{"erro":"Livro não encontrado com id: 99"}`.

---

3. Estrutura de pastas

```
BACKEND
├── pom.xml                              (lista de bibliotecas do projeto)
├── testes.http                          (os 7 testes de requisição)
└── src/main
    ├── resources
    │   └── application.properties       (configurações)
    └── java/com/biblioteca
        ├── BibliotecaApplication.java
        ├── CargaInicial.java
        ├── controller/LivroController.java
        ├── service/LivroService.java
        ├── repository/LivroRepository.java
        ├── entity/Livro.java
        ├── dto/LivroRequest.java
        ├── dto/LivroDTO.java
        ├── mapper/LivroMapper.java
        └── exception/
            ├── LivroNaoEncontradoException.java
            └── ApiExceptionHandler.java
```

---

4. Explicação arquivo por arquivo

4.1 `BibliotecaApplication.java` — o ponto de partida

```
package com.biblioteca;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class BibliotecaApplication {

    public static void main(String[] args) {
        SpringApplication.run(BibliotecaApplication.class, args);
    }
}
```

Linha	Explicação
`package com.biblioteca;`	Diz em qual "pasta" (pacote) esta classe está.
`import ...SpringApplication;`	Traz a classe que sabe **ligar** o Spring.
`import ...SpringBootApplication;`	Traz a anotação que marca a classe principal.

Linha	Explicação
`@SpringBootApplication`	Marca esta classe como a **principal**. Faz o Spring procurar, a partir deste pacote, todas as classes com anotações (`@RestController` , `@Service` , `@Entity` ...) e configurar tudo sozinho.
`public class BibliotecaApplication`	A classe principal.
`public static void main(String[] args)`	O `main` é onde **todo programa Java começa**.
`SpringApplication.run(...)`	Liga o servidor web (Tomcat embutido), cria o banco H2, cria as tabelas e deixa tudo pronto para receber pedidos.

Para explicar: *"É o botão de ligar. Rodar o `main` sobe o servidor na porta 8089."*

4.2 `entity/Livro.java` — o modelo que vira tabela

```
@Entity
public class Livro {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String titulo;
    private Boolean disponivel;

    private String capa;

    public Livro() {
    }

    public Livro(String titulo, Boolean disponivel, String capa) {
        this.titulo = titulo;
        this.disponivel = disponivel;
        this.capa = capa;
    }

    public Long getId() { return id; }
    public String getTitulo() { return titulo; }
    public void setTitulo(String titulo) { this.titulo = titulo; }
    public Boolean getDisponivel() { return disponivel; }
    public void setDisponivel(Boolean disponivel) { this.disponivel = disponivel; }
    public String getCapa() { return capa; }
    public void setCapa(String capa) { this.capa = capa; }
}
```

(No arquivo real os getters e setters ocupam várias linhas; aqui foram resumidos.)

Trecho	Explicação
`@Entity`	Diz ao JPA: "*esta classe é uma tabela do banco*". Vira a tabela `livro`, e cada objeto `Livro` é uma linha.
`@Id`	Marca o campo que é a **chave primária** (identifica cada livro de forma única).
`@GeneratedValue(strategy = GenerationType.IDENTITY)`	O **banco gera o número** sozinho (1, 2, 3...). Por isso nunca enviamos o id no cadastro.
`private Long id;`	O número do livro. `Long` é um número inteiro grande.
`private String titulo;`	O título (texto).
`private Boolean disponivel;`	`true` = disponível, `false` = indisponível. Usamos `Boolean` (com B maiúsculo) porque ele pode ser `null`, e é isso que permite a validação detectar "não enviou".
`private String capa;`	**Só o código** da capa (`aneis`, `harry`...). A imagem fica dentro do app Android, não no backend.
`public Livro() { }`	**Construtor vazio.** O JPA exige, porque ele cria o objeto vazio e preenche depois.
`public Livro(String titulo, ...)`	Construtor que usamos para criar um livro novo. **Não tem `id`**, porque o banco é quem cria.
`getX()` / `setX()`	Métodos para ler e alterar os campos (que são `private`). Não existe `setId`, porque o id não deve ser alterado.

Para explicar: *"A Entity é a ponte entre o Java e o banco. Cada campo vira uma coluna."*

4.3 `dto/LivroRequest.java` — o que o app envia (com validação)

```
public record LivroRequest(

        @NotBlank(message = "O título é obrigatório")
        @Size(max = 100, message = "O título pode ter no máximo 100 caracteres")
        String titulo,

        @NotNull(message = "A disponibilidade é obrigatória")
        Boolean disponivel,

        @NotBlank(message = "A capa é obrigatória")
        @Pattern(regexp = "aneis|harry|dom|orwel|principe",
                message = "Capa inválida. Use: aneis, harry, dom, orwel ou principe")
        String capa
) {
}
```

Trecho	Explicação
`record`	Um tipo de classe **curta e imutável** (não muda depois de criada). O Java cria sozinho o construtor e os métodos de leitura. Para ler um campo, usa-se `request.titulo()` (e não `getTitulo()`).
`@NotBlank`	O texto **não pode ser nulo, vazio ou só espaços**.
`@Size(max = 100)`	No máximo 100 caracteres.
`@NotNull`	O campo **precisa ser enviado** (não pode ser `null`). Usado no `disponivel`, porque `@NotBlank` só vale para texto.
`@Pattern(regexp = "aneis|harry|dom|orwel|principe")`	A capa só pode ser **uma** destas 5 palavras. O `|` significa "ou".
`message = "..."`	O texto do erro que volta para o app quando a regra falha.

Por que existe o `LivroRequest` e não usamos a Entity direto? Porque o cliente **não deve mandar o `id`**, e as regras de validação ficam em um lugar só, separado do banco.

Importante: as anotações só funcionam porque o Controller usa `@Valid`.

4.4 `dto/LivroDTO.java` — o que a API devolve

```
public record LivroDTO(Long id, String titulo, Boolean disponivel, String capa) {
}
```

É o "pacote de resposta". Tem o `id`, que só existe depois que o banco salva. O Spring transforma este objeto em JSON, por exemplo:

```
{"id":1,"titulo":"O Senhor dos Anéis","disponivel":true,"capa":"aneis"}
```

Para explicar: *"DTO = Data Transfer Object, o objeto que viaja entre as aplicações. Assim não expomos a Entity do banco."*

4.5 `mapper/LivroMapper.java` — o conversor

```
public class LivroMapper {

    public static Livro toEntity(LivroRequest request) {
        return new Livro(request.titulo(), request.disponivel(), request.capa());
    }

    public static LivroDTO toDTO(Livro livro) {
        return new LivroDTO(livro.getId(), livro.getTitulo(), livro.getDisponivel(),
livro.getCapa());
    }
}
```

Trecho	Explicação
`public static`	Método **estático**: dá para chamar direto, `LivroMapper.toDTO(...)`, sem criar um objeto `LivroMapper`.
`toEntity(LivroRequest request)`	Pega o que o app enviou e monta um `Livro` (sem id) para salvar no banco.
`toDTO(Livro livro)`	Pega um `Livro` do banco e monta o `LivroDTO` (com id) para responder.

Para explicar: "*Escrevemos o mapper à mão, sem biblioteca, para ficar fácil de entender: é só copiar campo por campo.*"

4.6 `repository/LivroRepository.java` — o acesso ao banco

```
public interface LivroRepository extends JpaRepository<Livro, Long> {
}
```

Trecho	Explicação
`interface`	Só declara o contrato; **não escrevemos o código**. O Spring cria a implementação sozinho.
`extends JpaRepository<Livro, Long>`	Herda vários métodos prontos. `Livro` = qual tabela; `Long` = o tipo do id.

Métodos prontos que usamos:

Método	O que faz
`findAll()`	Devolve todos os livros (`List<Livro>`)
`findById(id)`	Procura um livro pelo id. Devolve um `Optional<Livro>` (pode estar vazio)
`save(livro)`	Salva (insere) um livro e devolve o livro salvo, já com o id
`count()`	Conta quantos livros existem

Para explicar: "*Não escrevemos nenhum SQL. A interface está vazia e mesmo assim funciona.*"

4.7 `service/LivroService.java` — a lógica

```
@Service
public class LivroService {

    private final LivroRepository livroRepository;

    public LivroService(LivroRepository livroRepository) {
        this.livroRepository = livroRepository;
    }

    public List<LivroDTO> listarTodos() {
        return livroRepository.findAll().stream()
                .map(LivroMapper::toDTO)
                .toList();
    }

    public LivroDTO buscarPorId(Long id) {
        Livro livro = livroRepository.findById(id)
                .orElseThrow(() -> new LivroNaoEncontradoException(id));
        return LivroMapper.toDTO(livro);
    }

    public LivroDTO cadastrar(LivroRequest request) {
        Livro livro = LivroMapper.toEntity(request);
        Livro salvo = livroRepository.save(livro);
        return LivroMapper.toDTO(salvo);
    }
}
```

Trecho	Explicação
`@Service`	Marca a classe como a camada de **serviço**. O Spring cria **um** objeto dela e o entrega a quem precisar.
`private final LivroRepository livroRepository;`	O Service **usa** o Repository. `final` = depois de definido, não muda.
`public LivroService(LivroRepository livroRepository)`	**Injeção de dependência pelo construtor.** Não usamos `new`: o Spring entrega o Repository pronto.
`this.livroRepository = livroRepository;`	Guarda o Repository recebido no campo da classe.

`listarTodos()`

Trecho	Explicação
`livroRepository.findAll()`	Busca todos os `Livro` no banco.
`.stream()`	Transforma a lista em um "fluxo" para processar item a item.

Trecho	Explicação
`.map(LivroMapper::toDTO)`	Para **cada** `Livro`, chama `LivroMapper.toDTO`. O `::` é um atalho para "use este método".
`.toList()`	Junta o resultado de novo em uma lista de `LivroDTO`.

`buscarPorId(Long id)`

Trecho	Explicação
`findById(id)`	Procura o livro. Devolve um `Optional` (uma "caixa" que pode estar vazia).
`.orElseThrow(() -> new LivroNaoEncontradoException(id))`	Se a caixa tiver o livro, devolve ele. Se estiver vazia, **lança a exception**.
`return LivroMapper.toDTO(livro);`	Converte para DTO e devolve.

`cadastrar(LivroRequest request)`

Trecho	Explicação
`LivroMapper.toEntity(request)`	Converte o pedido em `Livro` (ainda sem id).
`livroRepository.save(livro)`	Salva no banco. O `salvo` volta **com o id** gerado.
`LivroMapper.toDTO(salvo)`	Converte para a resposta.

4.8 `controller/LivroController.java` — a porta de entrada

```
@RestController
@RequestMapping("/livros")
public class LivroController {

    private final LivroService livroService;

    public LivroController(LivroService livroService) {
        this.livroService = livroService;
    }

    @GetMapping
    public List<LivroDTO> listarTodos() {
        return livroService.listarTodos();
    }

    @GetMapping("/{id}")
    public LivroDTO buscarPorId(@PathVariable Long id) {
        return livroService.buscarPorId(id);
    }

    @PostMapping
    @ResponseStatus(HttpStatus.CREATED)
    public LivroDTO cadastrar(@RequestBody @Valid LivroRequest request) {
        return livroService.cadastrar(request);
    }
}
```

Trecho	Explicação
`@RestController`	Esta classe **recebe pedidos HTTP** e responde em **JSON**.
`@RequestMapping("/livros")`	Todos os métodos da classe começam com o endereço `/livros`.
`private final LivroService livroService;` + construtor	O Controller **usa** o Service (injeção pelo construtor, igual ao Service).
`@GetMapping`	Este método responde ao **GET** em `/livros`.
`listarTodos()`	Só chama o Service. **Não tem regra de negócio.**
`@GetMapping("/{id}")`	Responde ao **GET** em `/livros/algumNumero`.
`@PathVariable Long id`	Pega o número que veio **dentro do endereço** (`/livros/3` → `id = 3`).
`@PostMapping`	Responde ao **POST** em `/livros`.
`@ResponseStatus(HttpStatus.CREATED)`	A resposta de sucesso vai com **201** em vez do 200 padrão.
`@RequestBody`	O JSON que veio no corpo do pedido é transformado em um `LivroRequest`.

Trecho	Explicação
`@Valid`	**Liga a validação** (`@NotBlank` , `@Pattern` ...). Se falhar, o método **nem executa**.

Para explicar: *"O Controller só recebe e repassa. Quem pensa é o Service."*

4.9 `exception/LivroNaoEncontradoException.java` — o erro personalizado

```
public class LivroNaoEncontradoException extends RuntimeException {

    public LivroNaoEncontradoException(Long id) {
        super("Livro não encontrado com id: " + id);
    }
}
```

Trecho	Explicação
`extends RuntimeException`	É um tipo de erro do Java. Esse tipo **não obriga** a escrever `try/catch` ou `throws` em todo lugar.
Construtor com `Long id`	Recebe o id que não foi achado.
`super("... " + id)`	Passa a **mensagem do erro** para a classe-mãe. Fica `Livro não encontrado com id: 99` .

4.10 `exception/ApiExceptionHandler.java` — o tratador de erros

```
@RestControllerAdvice
public class ApiExceptionHandler {

    @ExceptionHandler(LivroNaoEncontradoException.class)
    @ResponseStatus(HttpStatus.NOT_FOUND)
    public Map<String, String> tratarNaoEncontrado(LivroNaoEncontradoException ex) {
        return Map.of("erro", ex.getMessage());
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    @ResponseStatus(HttpStatus.BAD_REQUEST)
    public Map<String, String> tratarValidacao(MethodArgumentNotValidException ex) {
        Map<String, String> erros = new LinkedHashMap<>();
        ex.getBindingResult().getFieldErrors()
                .forEach(erro -> erros.put(erro.getField(), erro.getDefaultMessage()));
        return erros;
    }

    @ExceptionHandler(HttpMessageNotReadableException.class)
    @ResponseStatus(HttpStatus.BAD_REQUEST)
    public Map<String, String> tratarJsonInvalido(HttpMessageNotReadableException ex) {
        return Map.of("erro", "JSON inválido ou campos com tipo errado");
    }
}
```

Trecho	Explicação
`@RestControllerAdvice`	Esta classe **vigia todos os Controllers**. Quando alguma exception escapa, ela decide a resposta.
`@ExceptionHandler(X.class)`	Este método trata **a exception do tipo X**.
`@ResponseStatus(...)`	O código HTTP que será devolvido.
`Map<String, String>`	Um mapa "chave → valor". O Spring transforma em JSON.

Os três tratadores

1. **`tratarNaoEncontrado`** → **404**. Devolve `{"erro":"Livro não encontrado com id: 99"}`. `ex.getMessage()` é o texto que definimos na exception.
2. **`tratarValidacao`** → **400**. Acontece quando o `@Valid` falha.
3. `new LinkedHashMap<>()` cria o mapa de erros (mantém a ordem).
4. `ex.getBindingResult().getFieldErrors()` lista os campos que falharam.
5. `.forEach(erro -> erros.put(erro.getField(), erro.getDefaultMessage()))` coloca no mapa **campo → mensagem**. Exemplo: `{"titulo":"O título é obrigatório"}`.
6. Se um campo falhar em mais de uma regra, o mapa guarda só uma mensagem para ele.
7. **`tratarJsonInvalido`** → **400**. Acontece quando o JSON está quebrado ou tem tipo errado, por exemplo `"disponivel": "abc"`. Devolve `{"erro":"JSON inválido ou campos com tipo errado"}`.

4.11 `CargaInicial.java` — os 4 livros iniciais

```
@Component
public class CargaInicial implements CommandLineRunner {

    private final LivroRepository livroRepository;

    public CargaInicial(LivroRepository livroRepository) {
        this.livroRepository = livroRepository;
    }

    @Override
    public void run(String... args) {
        if (livroRepository.count() == 0) {
            livroRepository.save(new Livro("O Senhor dos Anéis", true, "aneis"));
            livroRepository.save(new Livro("Harry Potter", false, "harry"));
            livroRepository.save(new Livro("Dom Casmurro", true, "dom"));
            livroRepository.save(new Livro("1984", false, "orwel"));
        }
    }
}
```

Trecho	Explicação
`@Component`	O Spring cria e gerencia esta classe.
`implements CommandLineRunner`	Promete ter um método `run`, que o Spring executa **uma vez, logo depois de ligar**.
`@Override`	Indica que estamos escrevendo o método `run` prometido pela interface.
`if (livroRepository.count() == 0)`	**Só cadastra se o banco estiver vazio.**
`livroRepository.save(new Livro(...))`	Salva cada livro. Ids gerados: 1, 2, 3, 4.

Como o banco fica na memória e é recriado a cada início, **os 4 livros sempre voltam** quando o servidor reinicia.

4.12 `application.properties` — as configurações

```
server.port=8089
spring.application.name=biblioteca

spring.datasource.url=jdbc:h2:mem:bibliotecadb
spring.datasource.username=sa
spring.datasource.password=
spring.jpa.hibernate.ddl-auto=create-drop
spring.jpa.open-in-view=false
```

Linha	Explicação
`server.port=8089`	O servidor escuta na porta **8089**. O app Android usa o mesmo número.
`spring.application.name=biblioteca`	Nome da aplicação.
`spring.datasource.url=jdbc:h2:mem:bibliotecadb`	Banco **H2 em memória** chamado `bibliotecadb`. Quando o servidor para, **os dados somem**.
`spring.datasource.username=sa`	Usuário padrão do H2.
`spring.datasource.password=`	Senha vazia.
`spring.jpa.hibernate.ddl-auto=create-drop`	Ao ligar, **cria** as tabelas (a partir da `@Entity`); ao desligar, **apaga**.
`spring.jpa.open-in-view=false`	Configuração de boa prática do Spring/JPA. Não muda o comportamento que mostramos.

4.13 `pom.xml` — as bibliotecas (Maven)

O Maven lê este arquivo e baixa as bibliotecas. As principais:

Dependência	Para que serve
`spring-boot-starter-web`	Criar a API web (servidor Tomcat embutido, Controllers, JSON)
`spring-boot-starter-data-jpa`	Falar com o banco usando objetos (JPA/Hibernate, Repository)
`spring-boot-starter-validation`	As anotações `@NotBlank`, `@NotNull`, `@Size`, `@Pattern`
`h2`	O banco de dados em memória

Versão do Spring Boot: **3.5.16**. Versão do Java configurada no projeto: **17**.

5. Os 7 testes (`testes.http`)

#	Pedido	Resultado esperado
1	`GET /livros`	200, lista com os livros
2	`GET /livros/1`	200, o livro 1
3	`GET /livros/99`	404, `{"erro":"Livro não encontrado com id: 99"}`
4	`POST` com dados válidos	201, o livro criado com id
5	`POST` com título vazio	400, `{"titulo":"O título é obrigatório"}`
6	`POST` sem `disponivel`	400, `{"disponivel":"A disponibilidade é obrigatória"}`
7	`POST` com capa `xyz`	400, `{"capa":"Capa inválida. Use: aneis, harry, dom, orwel ou principe"}`

Esses resultados foram conferidos rodando o servidor de verdade. Também foi conferido que um JSON com `"disponivel": "abc"` responde 400 com a mensagem de JSON inválido.

Como testar:

- Ligar o backend (IntelliJ → Run `BibliotecaApplication`).
- Usar o `testes.http` (no IntelliJ Community esse recurso não funciona; nesse caso use o `curl` ou o PowerShell).
- Exemplo no PowerShell: `Invoke-WebRequest http://localhost:8089/livros -UseBasicParsing`

---

6. Glossário rápido

Termo	Significado simples
API	O "cardápio" de pedidos que o servidor aceita
Endpoint	Um endereço específico da API (ex.: `/livros/{id}`)
HTTP	O protocolo (a "língua") dos pedidos pela rede
JSON	Formato de texto para trocar dados: `{"campo":"valor"}`
Spring Boot	Ferramenta que configura quase tudo sozinha para criar a API
Anotação (`@...`)	Uma "etiqueta" que diz ao Spring o que a classe ou o método faz
Injeção de dependência	O Spring entrega os objetos prontos, sem usarmos `new`
JPA / Hibernate	Ferramenta que liga objetos Java às tabelas do banco
Entity	Classe que vira tabela
DTO	Objeto que viaja entre as aplicações
Repository	Quem conversa com o banco
H2	Banco de dados simples, em memória
Bean Validation	As regras `@NotBlank`, `@NotNull`...
Exception	Um erro do programa
Optional	"Caixa" que pode ter ou não um valor

---

7. Perguntas que o professor pode fazer (com resposta curta)

1. **Por que separar em Controller, Service e Repository?** Cada camada tem uma responsabilidade. O código fica organizado, fácil de achar e de explicar.
2. **Por que usar DTO e não a Entity direto?** Para não expor o banco e para o cliente não mandar o `id`. A validação também fica no `LivroRequest`.
3. **O que o `@Valid` faz?** Liga as regras do `LivroRequest`. Se alguma falhar, o Spring lança `MethodArgumentNotValidException` e o `ApiExceptionHandler` responde 400.
4. **O que acontece se eu pedir um livro que não existe?** O Service lança `LivroNaoEncontradoException`, e o handler responde 404.
5. **Por que 201 no POST?** 201 significa "criado". É o código correto quando um recurso novo é criado.

6. **Os dados ficam salvos?** Não. O H2 está em memória; ao reiniciar, voltam só os 4 livros iniciais.
7. **Por que a capa é só um texto?** O backend guarda só o código. A imagem fica dentro do app Android. Isso evita upload de imagens.
8. **O que é injeção de dependência?** O Spring cria os objetos e os entrega pelo construtor. A gente não escreve `new LivroService(...)`.
9. **Como o Spring achou meus Controllers?** O `@SpringBootApplication` faz o Spring varrer os pacotes a partir de `com.biblioteca`.
10. **Por que não tem PUT e DELETE?** O escopo do trabalho é somente listar, buscar e cadastrar.

---

8. Roteiro sugerido para a apresentação (5 a 8 minutos)

1. **Contexto:** biblioteca com 4 livros; o app Android consome esta API.
2. **Mostrar o desenho das camadas** (a seção 2) e explicar o caminho de um POST.
3. **Mostrar o código na ordem:** `Livro` → `LivroRequest` → `Repository` → `Service` → `Controller` → exceptions.
4. **Demonstração:** ligar o servidor; `GET /livros`; `GET /livros/99` (404); um `POST` válido; um `POST` com título vazio (400).
5. **Fechar:** explicar que o banco é em memória e que o app Android usa estes mesmos endpoints.

Checklist antes de apresentar

☐ Reiniciar o backend para o banco ter só os 4 livros iniciais.
☐ Confirmar que está na porta **8089**.
☐ Ensaiar uma vez o `GET`, o `404` e o `POST`.